

Microservice Scalable Institution Student Portal

Proposed Services & Execution Plan

Akwenwi Kenzel Che Niba | ICT University, Yaoundé

Prepared 16 September 2026 | Target delivery: 1 October 2026

Scope reality check. This document is written against a 15-day window (16 Sep – 1 Oct). The service catalogue in Part 1 is deliberately larger than what fits in that window, and every entry is tagged **Core**, **Stretch**, or **Post-deadline**. Part 2 schedules only the Core set. Attempting the full catalogue before 1 October would put the one gradeable claim — 1,000 concurrent users without crashing — at risk, which is the opposite of what a strong defence needs.

Contents

- **Part 1** — Proposed services: the full catalogue, tiered by priority
- **Part 2** — Step-by-step execution plan, 16 September to 1 October
- **Part 3** — Defence deliverables checklist
- **Part 4** — Risk register and cut-list

Part 1 — Proposed Services

The original plan defined six services: API Gateway, Auth, Student, Dashboard/BFF, Transcript, Past Papers, and Notification. What follows keeps those and adds the services that would make the portal complete as a real product. Tier tags indicate what is achievable before 1 October versus what belongs in a post-graduation roadmap.

1.1 Tier definitions

Tier	Meaning
Core	Required for a defensible final-year submission. Scheduled in Part 2.
Stretch	Build only if Core is finished and load-tested early. Adds visible polish.
Post-deadline	Genuinely valuable, but not buildable in 15 days. Present these in your report as a documented roadmap — examiners credit a candidate who knows what they deliberately deferred.

1.2 Services already in the original plan

Service	Responsibility	Tier
API Gateway	Single entry point, routing, rate limiting, token termination. Traefik.	Core
Auth Service	Signup/login, JWT issue and refresh, roles (student/lecturer/admin), password reset.	Core
Student Service	Profile, enrolment status, programme and department data.	Core
Dashboard / BFF	Aggregates other services into one payload for the frontend home screen. No DB of its own.	Core
Transcript Service	Grades, GPA calculation, PDF transcript export via WeasyPrint.	Core
Past Papers Service	Upload, store, search and download past exam papers by course/year/semester; MinIO + Postgres metadata.	Core
Notification Service	Consumes RabbitMQ events, sends email/SMS. Minimal version for the deadline: one event type, one template.	Core (minimal)

1.3 Proposed additions — high value

These are the gaps most visible to an examiner. A student portal without course registration or fee visibility is noticeably incomplete as a product, even when the architecture is sound.

Service	What it does	Why it matters	Tier
Course & Enrolment Service	Course catalogue, prerequisites, semester registration, add/drop, class lists.	The single biggest functional gap. Transcript grades logically depend on enrolled courses, so it also strengthens your data model.	Stretch
Timetable Service	Class schedules, exam timetables, room allocation, personal timetable per student.	High perceived value for very little architectural risk — mostly CRUD plus a calendar view. Good demo material.	Stretch
Fees & Payments Service	Fee structure, balance, payment history, receipts. Mobile-money integration (MTN MoMo / Orange Money) via sandbox.	Locally relevant and a strong differentiator at defence. Use sandbox credentials only — never live payment flows in a student project.	Post-deadline
Announcements Service	Institution-wide and course-level notices, pinned items, read receipts.	Pairs naturally with your existing Notification Service and RabbitMQ event bus — low extra cost, visible payoff.	Stretch

Service	What it does	Why it matters	Tier
Admin / Institution Service	Multi-tenant management: institution onboarding, per-tenant config, admin user management.	You already committed to institution_id on every table. This is the service that makes the SaaS claim real rather than theoretical.	Post-deadline

1.4 Proposed additions — infrastructure and quality

These do not add user-facing features, but several of them directly produce the evidence your defence depends on.

Service	What it does	Why it matters	Tier
Observability stack (Prometheus + Grafana)	Live metrics: request rate, latency percentiles, error rate, DB connection pool usage.	Upgraded from 'nice to have' to Core . A Grafana panel showing the system holding steady at 1,000 users is far more persuasive than a k6 text summary, and it is how you will actually find your bottleneck.	Core
Centralised logging (Loki or ELK)	Aggregates logs from every service into one searchable place.	With seven services, debugging by running docker compose logs per container becomes painful fast. Loki is the lighter option and pairs with Grafana you already have.	Stretch
Search Service (Meilisearch)	Full-text search across past papers, courses, announcements.	Postgres full-text search covers the deadline version. Meilisearch is the upgrade if Past Papers search feels slow under load.	Post-deadline
Audit Service	Immutable log of sensitive actions: grade changes, role changes, document downloads.	Strong answer to the examiner question 'how do you prevent a lecturer silently altering a grade?' Consumes existing RabbitMQ events, so it is additive rather than invasive.	Stretch
API documentation (Swagger / OpenAPI)	Per-service interactive API docs, aggregated at the gateway.	Cheap to add per service if done as you build. Doubles as report appendix material.	Core
Reporting / Analytics Service	Admin dashboards: enrolment numbers, grade distributions, resource usage.	Gives the admin role a reason to exist beyond user management.	Post-deadline

1.5 Recommended cut for the 1 October build

If you build exactly this and prove it at scale, you have a complete, defensible submission:

- All seven original services, with Notification kept minimal
- Prometheus + Grafana (promoted to Core — it is your evidence)
- Swagger docs written per service as you go, not retrofitted
- **One** stretch service if Core finishes early — Timetable is the cheapest to build; Course & Enrolment is the most impressive

Recommendation. Do not start a stretch service before your 1,000-user load test passes. An extra feature adds a few marks; a failed core claim costs far more. Everything in the Post-deadline tier goes into your report as a roadmap section, which costs you nothing and demonstrates judgement.

Part 2 — Step-by-Step Execution Plan

Fifteen days, 16 September to 1 October. The sequencing rule throughout: **prove the scalable core first, then layer features on it**. Each phase ends with something demonstrably working, so that if time runs out you always have a working system rather than several half-finished ones.

Phase 0 — Unblock the pipeline (16 September, half a day)

The gateway must route before anything else is worth building.

- Run `.\diagnose-socket.ps1` and read Test 3.
- If Test 3 fails: Docker Desktop → Settings → Advanced → enable **Allow the default Docker socket to be used**; confirm the WSL 2 engine is on under Settings → General; fully **Quit** Docker Desktop from the system tray and relaunch.
- Re-run the diagnostic; Test 3 should return JSON.
- `docker compose down; docker compose up -d --build`
- Confirm: `curl http://localhost/health` returns JSON, and the health router appears at `localhost:8080/dashboard/`.

Exit criterion: a request entering the gateway reaches a container and returns. Do not start Auth until this passes.

Phase 1 — Auth Service (16–18 September)

Everything else depends on Auth, so it is built and load-tested first, alone.

- Design the auth schema: users, roles, refresh tokens — each table carrying `institution_id` from day one.
- Build endpoints: register, login, refresh, logout, password reset request/confirm.
- Hash passwords with `bcrypt` or `argon2`. Never store plaintext; never log tokens.
- Connect via **PgBouncer** (`pgbouncer : 6432`), not Postgres directly. This is the difference between surviving and crashing at 1,000 users.
- Store refresh tokens/sessions in Redis.
- Add Traefik labels for `/auth`; add the service to the CI matrix.
- Write Swagger docs for these endpoints now, while they are fresh.
- **Write and run your first k6 script against Auth alone.** Record the baseline numbers.

Exit criterion: a user can register and log in through the gateway, and you have a k6 baseline report for Auth at 100 and 500 virtual users.

Phase 2 — Student Service + Dashboard/BFF (19–21 September)

- Student Service: profile, enrolment status, programme/department. Own schema, own role.
- Dashboard/BFF: calls Auth and Student, composes one payload. No database of its own.
- Cache the dashboard aggregate in Redis — it is your most-hit read path, so this is where caching pays.
- Frontend: login flow plus a dashboard shell rendering real data.
- Load test Auth + Student + BFF together, ramping 100 → 500 → 750.
- Stand up **Prometheus + Grafana** now, not at the end — you need the graphs to interpret Phase 4, and configuring monitoring under deadline pressure later is a predictable way to lose an evening.

Exit criterion: logging in shows a populated dashboard, and Grafana displays live latency and error-rate panels during a load test.

Phase 3 — Transcript + Past Papers (22–25 September)

Two services in four days. Transcript first — it is the harder of the two.

- **Transcript Service:** grades, GPA calculation, PDF export with WeasyPrint (experience you already have). Own schema.
- Publish a `grade . posted` event to RabbitMQ on new grades.
- **Past Papers Service:** MinIO for the files, Postgres for metadata (course, year, semester, uploader, tags). Postgres full-text search is sufficient — do not reach for Meilisearch before the deadline.
- Publish a `pastpaper . up loaded` event.
- **Notification Service, minimal:** consume those two events, send one email each. Resist expanding this.
- Frontend: transcript view/download, Past Papers browse/upload/download.
- Swagger docs for both services as you build them.

Exit criterion: a student can view and download a PDF transcript, and upload/search/download a past paper — all through the gateway.

Phase 4 — Full-system scale test (26–28 September)

This phase produces the evidence your entire claim rests on. Treat it as real work with real debugging time, not a formality at the end.

- **Run the load generator from a separate machine** — never on the box under test, or your numbers are meaningless.
- Ramp deliberately: 250 → 500 → 750 → 1,000 concurrent. Record response time percentiles (p50/p95/p99) and error rate at each step.
- Watch Grafana *during* the run. The panel that saturates first is your bottleneck.
- Expect to find at least one real bottleneck. The usual suspects, in order: Postgres connection exhaustion (check PgBouncer pool settings), missing cache on a hot read path, and under-replicated services.
- Scale services horizontally: `docker compose up -d --scale auth-service=3`. Remove `container_name` from any service you scale.
- Re-run until 1,000 concurrent holds with an acceptable error rate. **Then** try 3,000 as a bonus — but 1,000 is the requirement, and a clean 1,000 beats a messy 3,000.
- **Save every k6 report and Grafana screenshot.** These are graded artefacts.

Decision point, 28 September. If 1,000 concurrent passes cleanly and you have spare time, this is the only moment to consider a stretch service. If it does not pass yet, cut all remaining scope and fix the bottleneck. The scalability claim is the project.

Phase 5 — Documentation, polish, rehearsal (29–30 September)

- **Feature freeze on 29 September.** No new functionality after this point, regardless of temptation.
- UI polish pass across every section — an area where your ByKenzel eye is a genuine advantage.
- Write up: architecture diagrams, service responsibilities, database strategy (schema-per-service and why), storage split (Cloudinary vs MinIO and why), and the load-test results with graphs.
- Include the **Post-deadline roadmap** from Part 1 — deliberate deferral reads as judgement, not omission.
- **Record a full demo video as a backup** in case the live demo fails on the day. Do not skip this.
- Rehearse the walkthrough end to end, out loud, at least twice.
- Prepare answers for the predictable questions: Why microservices? Why schema-per-service rather than database-per-service? How do you prevent a lecturer silently altering a grade? What happens when one service goes down?

Phase 6 — Final checks (1 October)

- Clean deployment from scratch on the Oracle Cloud instance — prove it builds from a fresh clone, not just on your laptop.
- Verify every route through the gateway one final time.
- Confirm the demo video, k6 reports, Grafana screenshots and written report are all in hand before you walk in.

Part 3 — Defence Deliverables Checklist

What you must physically have on defence day, independent of whether the live demo cooperates.

Deliverable	Produced in	Status
Working deployed system reachable through the gateway	Phases 1–3	■
k6 load-test reports at 250 / 500 / 750 / 1,000 concurrent users	Phase 4	■
Grafana screenshots showing system health during the 1,000-user run	Phase 4	■
Architecture diagram (final, matching what you actually built)	Phase 5	■
Database schema documentation, with the schema-per-service rationale	Phase 5	■
Swagger/OpenAPI docs per service	Phases 1–3	■
Written report including the deferred-services roadmap	Phase 5	■
Demo video backup	Phase 5	■
Clean-clone deployment verified on Oracle Cloud	Phase 6	■

The k6 reports and Grafana screenshots are the highest-value items on this list. They convert 'the system is scalable' from an assertion into measured evidence — which is precisely the distinction examiners probe.

Part 4 — Risk Register and Cut-List

4.1 Known risks

Risk	Mitigation
Postgres connection exhaustion — the single most likely cause of a crash at 1,000 users.	PgBouncer in transaction mode, in front of Postgres, from day one. Every service connects through port 6432, never 5432 directly. Tune DEFAULT_POOL_SIZE during Phase 4.
Building services before load-testing any of them — discovering a bottleneck on 28 September is far worse than on 18 September.	Load test Auth alone in Phase 1. Test cumulatively at every phase boundary. Never let more than one untested service accumulate.
Load generator on the same machine as the system under test — produces skewed, indefensible numbers.	Run k6 from a separate machine or a second free-tier instance. State this explicitly in the report; examiners notice.
Oracle Cloud Always Free capacity limits — 4 ARM CPUs and 24 GB RAM is generous but finite.	Monitor resource usage in Grafana during Phase 4. If you saturate the host, say so honestly in the report and document what capacity would be needed — that is a legitimate, creditable finding.
Scope creep from the stretch services — the most likely way this deadline is missed.	Hard rule: no stretch service starts before the 1,000-user test passes on 28 September.
Single-developer illness or a lost day — 15 days has no natural slack.	The plan front-loads the critical path deliberately. If a day is lost, cut from Phase 3 (reduce Past Papers to upload and list only) before cutting anything from Phase 4.

4.2 Cut-list, in the order things should be dropped

If you fall behind, cut in this sequence. Decide by the listed date rather than deferring the decision.

- **1.** All stretch services — automatic, unless Phase 4 passes early.

- **2.** Centralised logging — docker compose logs is survivable for a demo.
- **3.** Past Papers search and tagging — reduce to upload and list only (decide by 25 September).
- **4.** Notification Service — reduce to a logged event rather than a real email (decide by 25 September).
- **5.** The 3,000-user test — 1,000 is the stated requirement; 3,000 is a bonus.
- **6.** UI polish — functional beats beautiful when time is gone.

Never cut: PgBouncer, the 1,000-concurrent load test and its evidence, or the demo video backup. Those three are what the submission stands or falls on.

Prepared for Akwenwi Kenzel Che Niba — Final Year Project, ICT University, Yaoundé. Timeline assumes a 16 September start and 1 October delivery.